Week 1 Module: Programming practice (3rd year)

- Language Used: Java

Star Pattern

1. Square

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to control the number of rows to be printed.
4. For each row, use an inner loop to print * symbols equal to the number of rows.
5. After the inner loop completes, print a new line to move to the next row.
6. Repeat the process until all rows are printed, forming a square pattern.
7. End the program.

Source Code

```
import java.util.Scanner;
public class Square {
    void main() {
      try(Scanner scan = new Scanner(System.in)){
         System.out.print("Enter input: ");
         int rows = scan.nextInt();
         for (int i = 0; i < rows; i++) {
            for (int j = 0; j < rows ; j++) {
               System.out.print("*");
            }
            System.out.print("\n");
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

```
Enter input: 5
*****
*****
*****
*****
*****

Process finished with exit code 0
```

2. Hollow-Square

```
* * * * *
*       *
*       *
*       *
* * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row.
4. For each row:
    - If it is the **first** or **last** row, print * for all columns to form the top and bottom borders.
    - Otherwise, use an inner loop to iterate through each column:
        - Print * if it is the **first** or **last** column.
        - Print a space " " for all other columns to create the hollow portion.
5. After printing each row, move to the next line.
6. Repeat until all rows are printed, forming a hollow square pattern.
7. End the program.

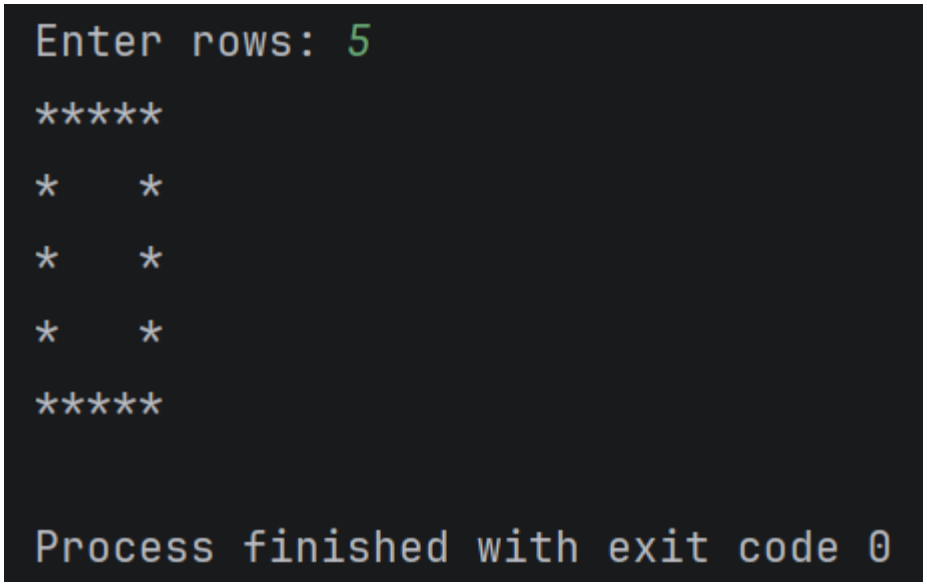
Source Code

```
import java.util.Scanner;

class HollowSquare {

   static void main(String[] args) {
      try(Scanner scan = new Scanner(System.in)){
         System.out.print("Enter rows: ");
         int rows = scan.nextInt();
         for (int i = 0; i < rows; i++) {
            if (i==0 || i == rows-1) for (int j = 0; j < rows; j++) System.out.print("*");
            else{
               for (int j = 0; j < rows ; j++) {
                  if(j == 0 || j == rows-1) System.out.print("*");
                  else System.out.print(" ");
               }
            }
            System.out.println();
         }
      }
catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output



3. Triangle

```
*
* *
* * *
* * * *
* * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row.
4. For each row, use an inner loop to print * symbols equal to the current row number (i + 1).
5. After printing the stars for a row, move to the next line.
6. Repeat the process until all rows are printed, forming a right-angled triangle pattern.
7. End the program.

Source Code

```
import java.util.Scanner;

class Triangle{

   void main(){
      try (Scanner scan = new Scanner(System.in)){
         System.out.print("Enter rows: ");
         int rows = scan.nextInt();
         for (int i = 0; i < rows; i++) {
            for (int j = 0; j <= i; j++) {
               System.out.print("*");
            }
            System.out.println();
         }
      }
catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

```
Enter rows: 5
*
**
***
****
*****

Process finished with exit code 0
```

4. Inverted-Triangle

```
* * * * *
* * * *
* * *
* *
*
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row.
4. For each row, use an inner loop to print * symbols. The number of stars starts from the total number of rows and decreases by one in each subsequent row.
5. After printing the stars for a row, move to the next line.
6. Repeat the process until all rows are printed, forming an inverted right-angled triangle pattern.
7. End the program.

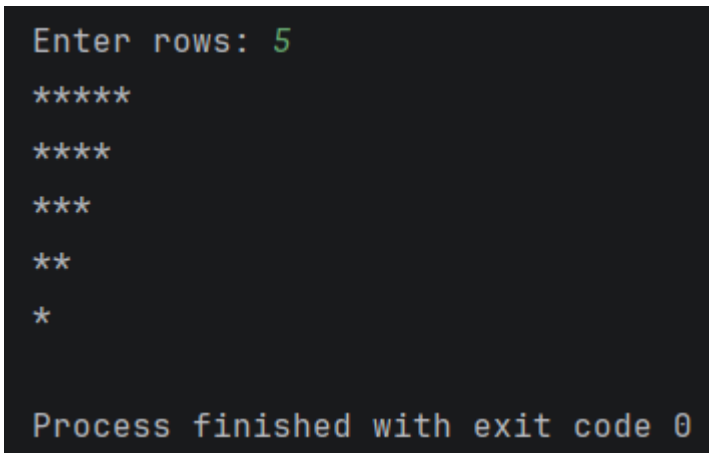
Source Code

```
import java.util.Scanner;

class InvertedTriangle{
   void main(){
      try (Scanner scan = new Scanner(System.in)){
         System.out.print("Enter rows: ");
         int rows = scan.nextInt();
```

```
        for (int i = 0; i < rows; i++) {
            for (int j = rows; j > i; j--) {
                System.out.print("*");
            }
            System.out.println();
        }
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
  }
}
```

Output



5. Pyramid

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row from 1 to the input value.
4. For each row:
    - Print the required number of leading spaces so that the stars are centered. The number of spaces decreases with each row.

- Print 2 × row − 1 stars using an inner loop. The number of stars increases by two in every successive row.

5. After printing the spaces and stars for a row, move to the next line.
6. Repeat the process until all rows are printed, forming a pyramid pattern.
7. End the program.

Source Code

```
import java.util.Scanner;

class Pyramid {

   void main(){

      try (Scanner scan = new Scanner(System.in)){
         System.out.print("Enter input: ");
         int input = scan.nextInt();
         for (int i = 1; i <= input; i++) {
            for (int space = input; space > i; space--) {
               System.out.print(" ");
            }



            for (int j = 0; j < (2 * i - 1); j++) {
               System.out.print("*");
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

```
Enter input: 5
    *
   ***
  *****
 *******
*********

Process finished with exit code 0
```

6. Diamond

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Print the **upper half** of the diamond:
    - Use an outer loop to iterate from 1 to the input value.
    - For each row, print the required leading spaces to centre the pattern.
    - Print 2 × row − 1 stars to form the pyramid.
    - Move to the next line.
4. Print the **lower half** of the diamond:
    - Use another outer loop to iterate from the input value down to 1.
    - For each row, print the required leading spaces, increasing with each iteration.
    - Print 2 × row − 1 stars, decreasing with each iteration.
    - Move to the next line.

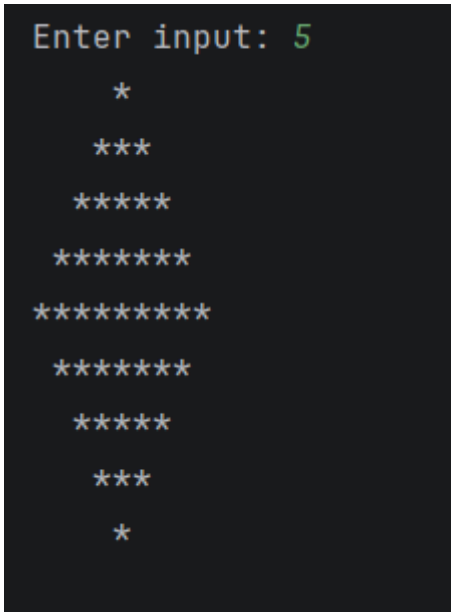
5. Repeat until both halves are printed, forming a complete diamond pattern.
6. End the program.

Source Code

```
import java.util.Scanner;

public class Diamond {
   void main(){
      try (Scanner scan = new Scanner(System.in)){
         System.out.print("Enter input: ");
         int input = scan.nextInt();
         for (int i = 1; i <= input; i++) {
            for (int space = input; space > i ; space--) {
               System.out.print(" ");
            }
            for (int j = 0; j < 2*i-1; j++) {
               System.out.print("*");
            }
            System.out.println();
         }
         for (int i = input; i > 0; i--) {
            for (int space = 0; space < input -i ; space++) {
               System.out.print(" ");
            }
            for (int j = 0; j < (2 * i - 1); j++) {
               System.out.print("*");
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output



7. Half-Diamond

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Print the **upper half** of the pattern:
    - Use an outer loop to iterate from the first row to the specified number of rows.
    - For each row, use an inner loop to print stars equal to the current row number (i + 1).
    - Move to the next line after printing each row.
4. Print the **lower half** of the pattern:
    - Use another outer loop to iterate from rows - 1 down to 1.
    - For each row, use an inner loop to print stars equal to the current loop value.
    - Move to the next line after printing each row.

5. Repeat until both halves are printed, forming a half-diamond pattern.
6. End the program.

Source Code

```
import java.util.Scanner;

class HalfDiamond{
   void main(){
      try (Scanner scan = new Scanner(System.in)){
         System.out.print("Enter input: ");
         int rows = scan.nextInt();
         for (int i = 0; i < rows; i++) {
            for (int j = 0; j <= i; j++) {
               System.out.print("*");
            }
            System.out.println();
         }
         for (int i = rows-1; i > 0 ; i--) {
            for (int j = 0; j < i; j++) {
               System.out.print("*");
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

8. Half-Diamond-Inverted

```
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

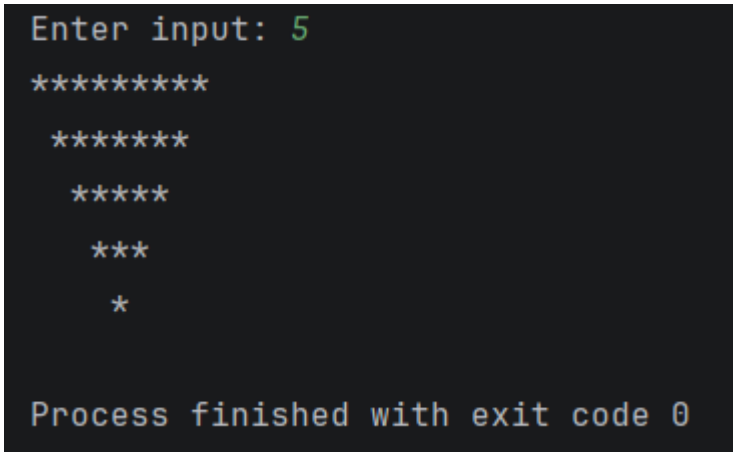
Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate from the input value down to 1.
4. For each row:
    - Print the required number of leading spaces. The number of spaces increases by one in each successive row.
    - Print 2 × row − 1 stars using an inner loop. The number of stars decreases by two in each successive row.
5. After printing the spaces and stars for a row, move to the next line.
6. Repeat the process until all rows are printed, forming an inverted pyramid pattern.
7. End the program.

Source Code

```
import java.util.Scanner;
public class InvertedPyramid {
   void main(){
      try (Scanner scan = new Scanner(System.in)){
         System.out.print("Enter input: ");
         int input = scan.nextInt();
         for (int i = input; i > 0; i--) {
            for (int space = 0; space < input-i ; space++) {
               System.out.print(" ");
            }
            for (int j = 0; j < (2 * i - 1); j++) {
               System.out.print("*");
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output



Number Pattern

1. Square

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row.
4. For every row, use an inner loop to print the digit 1 exactly equal to the number of rows.
5. After printing one row, move to the next line.
6. Repeat the process until all rows are printed, forming a square of 1s.
7. End the program.

Source Code

```
import java.util.Scanner;

public class NumberSquare {
   void main(){
      try(Scanner scan = new Scanner(System.in)){
         System.out.print("Enter rows: ");
         int rows = scan.nextInt();
```

```
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < rows; j++) {
                System.out.print("1");
            }
            System.out.println();
        }
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
  }
}
```

Output

```
Enter rows: 5
11111
11111
11111
11111
11111

Process finished with exit code 0
```

2. Incrementing

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row.
4. For each row, use an inner loop to print numbers from 1 to the specified number of rows in increasing order.
5. After printing all the numbers in a row, move to the next line.
6. Repeat the process until all rows are printed, forming a square pattern with incrementing numbers in each row.

7. End the program.

Source Code

```
import java.util.Scanner;

public class IncrementingSquare {
   void main() {
      try(Scanner input = new Scanner(System.in)) {
         System.out.print("Enter rows: ");
         int rows = input.nextInt();
         for (int i = 0; i < rows; i++) {
            for (int j = 1; j <= rows; j++) {
               System.out.print(j);
            }
            System.out.println();
         }
      }
   }
}
```

Output

```
Enter rows: 5
12345
12345
12345
12345
12345

Process finished with exit code 0
```

3. Right-triangle

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Logic

1. Start the program.

2. Read the number of rows from the user.
3. Use an outer loop to iterate from 1 to the specified number of rows.
4. For each row, use an inner loop to print numbers starting from 1 up to the current row number in increasing order.
5. After printing the numbers for a row, move to the next line.
6. Repeat the process until all rows are printed, forming a right-angled triangle of numbers.
7. End the program.

Source Code

```
import java.util.Scanner;

public class RightTriangle {
   void main(String args[]){
      try(Scanner sc=new Scanner(System.in)) {
         System.out.print("Enter input: ");
         int n = sc.nextInt();
         for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
               System.out.print(j);
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

4. Inverted-Diamond

```
5 5 5 5 5 5 5 5 5
  4 4 4 4 4 4 4
    3 3 3 3 3
      2 2 2
        1
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate from the input value down to 1.
4. For each row:
    - Print the required number of leading spaces. The number of spaces increases by one in each successive row.
    - Print the current row number (i) exactly 2 × i − 1 times using an inner loop.
5. After printing the spaces and numbers for a row, move to the next line.
6. Repeat the process until all rows are printed, forming an inverted number pyramid pattern.
7. End the program.

Source Code

```
import java.util.Scanner;

public class InvertedNumberPyramid {
   void main(String args[]){
      try(Scanner sc=new Scanner(System.in)) {
         System.out.print("Enter input: ");
         int n = sc.nextInt();
         for (int i = n; i >= 1; i--) {
            for (int space = 0; space < n-i; space++) {
               System.out.print(" ");
            }
            for (int j = 1; j <= 2*i-1 ; j++) {
               System.out.print(i);
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

```
Enter input: 5
555555555
 4444444
  33333
   222
    1

Process finished with exit code 0
```

5. Sandwich-Pattern

```
5 5 5 5 5 5 5 5 5
  4 4 4 4 4 4 4
    3 3 3 3 3
      2 2 2
        1
      2 2 2
    3 3 3 3 3
  4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Print the **upper half** of the pattern:
    - Use an outer loop to iterate from n down to 2.
    - For each row, print the required leading spaces. The number of spaces increases as the row number decreases.
    - Print the current row number (i) exactly $2 \times i - 1$ times.
    - Move to the next line.
4. Print the **lower half** of the pattern:
    - Use another outer loop to iterate from 1 to n.
    - For each row, print the required leading spaces. The number of spaces decreases as the row number increases.
    - Print the current row number (i) exactly $2 \times i - 1$ times.

- Move to the next line.

5. Repeat until both halves are printed, forming a symmetric number pattern with the largest number at the centre.
6. End the program.

Source Code

```
import java.util.Scanner;

public class Sandwich {
   void main(String args[]){
      try(Scanner sc=new Scanner(System.in)) {
         System.out.print("Enter input: ");
         int n = sc.nextInt();
         for (int i = n; i >= 2; i--) {
            for (int space = 0; space < n-i; space++) {
               System.out.print(" ");
            }
            for (int j = 1; j <= 2*i-1 ; j++) {
               System.out.print(i);
            }
            System.out.println();
         }
         for (int i = 1; i <= n; i++) {
            for ( int space = i; space <= n-1; space++) {
               System.out.print(" ");
            }
            for (int j = 0; j < i*2-1; j++) {
               System.out.print(i);
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

```
Enter input: 5
555555555
 4444444
  33333
   222
    1
   222
  33333
 4444444
555555555

Process finished with exit code 0
```

6. Incrementing-Diamond

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Print the **upper half** of the pattern:
    - Use an outer loop to iterate from 1 to n.
    - For each row, use an inner loop to print the current row number (i) exactly i times.
    - Move to the next line after printing each row.
4. Print the **lower half** of the pattern:
    - Use another outer loop to iterate from n - 1 down to 0.
    - For each row, use an inner loop to print the current row number (i) exactly i times.
    - Move to the next line after printing each row.
5. Repeat until both halves are printed, forming a half-diamond number pattern.

6. End the program.

Source Code

```
import java.util.Scanner;

public class HalfDiamondPattern {
   void main(String args[]){
      try(Scanner sc=new Scanner(System.in)) {
         System.out.print("Enter input: ");
         int n = sc.nextInt();
         for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
               System.out.print(i);
            }
            System.out.println();
         }
         for (int i = n-1; i >= 0; i--) {
            for (int j = 1; j <= i; j++) {
               System.out.print(i);
            }
            System.out.println();
         }
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }
}
```

Output

```
Enter input: 5
1
22
333
4444
55555
4444
333
22
1


Process finished with exit code 0
```